

Chapter 26: Content Projection

1. Overview

Content projection is Angular's mechanism for letting a parent template pass markup *into* a child component's template, where the child decides where and how that markup renders. It is Angular's answer to the DOM's native `<slot>` mechanism (Web Components), and it is the foundation for every "container" or "wrapper" component you will ever build: cards, modals, tabs, accordions, dialogs, panels, list items, form field wrappers, and so on.

The core directive is `<ng-content>`. Unlike `*ngIf` / `*ngFor`, `<ng-content>` is not a structural directive and does not create an embedded view with its own change-detection context — it is a **placeholder marker** that Angular's compiler resolves at compile time into a projection instruction. The content that fills it is **not** rendered by the child component; it is rendered by the parent, and merely *moved* (logically, not physically re-parsed) into the child's DOM position.

Why this matters for interviews: almost every question in this chapter reduces to one fact: **projected content belongs to, and is checked against, the component that declared it — never the component that receives it**. Once you internalize that, gotchas around bindings, `this`, and change detection stop being surprising.

This chapter covers:

- Single-slot and multi-slot projection (`<ng-content select="...">`)
- `ngProjectAs` for aligning custom elements/attributes with a `select` query
- Conditional and dynamic projection (`*ngIf` around `<ng-content>`, `ngTemplateOutlet`, multiple named slots toggled by state)
- `@ContentChild` / `@ContentChildren` — querying the *projected* content from within the host component
- Building production-grade reusable Card, Tabs, and Modal components

2. Core Concepts

2.1 What `<ng-content>` actually is

`<ng-content>` is an Angular-specific element recognized by the compiler. It never appears in the rendered DOM — Angular strips the `<ng-content>` tag itself and replaces it with the projected nodes (or nothing, if no matching content was supplied and no fallback content exists).

```
<!-- card.component.html -->
<div class="card">
  <ng-content></ng-content>
</div>
```

```
<!-- usage -->
<app-card>
  <p>This paragraph is projected into the card.</p>
</app-card>
```

Rendered DOM:

```
<app-card>
  <div class="card">
    <p>This paragraph is projected into the card.</p>
  </div>
</app-card>
```

Everything between `<app-card>` and `</app-card>` in the parent template is the **projectable content**. It is parsed as part of the *parent's* template, type-checked against the *parent's* component class, and only relocated into the child's view at render time.

2.2 Single-slot projection

With a single unqualified `<ng-content>`, **all** content passed by the parent is projected into that one location, regardless of how many elements or what they are:

```
<ng-content></ng-content>
```

This is the simplest case — a "transparent wrapper" component (e.g., a styled `<div>` wrapper, a tooltip host, a generic panel).

2.3 Multi-slot projection with `select`

To split incoming content into distinct buckets, use multiple `<ng-content>` elements, each with a `select` attribute containing a CSS selector. Angular walks the *direct children* of the host's projectable content and assigns each top-level node to the **first** matching `<ng-content select="...">` it encounters, in **document order of the `<ng-content>` tags in the template**, not in the order content was authored by the parent.

```
<!-- card.component.html -->
<div class="card">
  <div class="card-header">
    <ng-content select="[card-header]"></ng-content>
  </div>
  <div class="card-body">
    <ng-content></ng-content> <!-- catch-all: unmatched content lands here -->
  </div>
  <div class="card-footer">
    <ng-content select="[card-footer]"></ng-content>
  </div>
</div>
```

Selector types supported by `select`:

- Element selector: `select="h1"`

- Attribute selector: `select="[card-header]"`
- Class selector: `select=".title"`
- Combined: `select="h1[card-header].title"`
- Comma-separated (matches any): `select="h1, .title"`

The unqualified `<ng-content>` (no `select`) acts as a **wildcard/catch-all**: it matches any projected node that was not claimed by an earlier, more specific `<ng-content select>`. Placement of the wildcard `<ng-content>` relative to the selective ones in the template only affects *where in the DOM* the leftover content appears, not *which* content it catches — matching is selector-driven, not position-driven, except that the wildcard only picks up what nothing else claimed.

If you omit the wildcard entirely, any content that doesn't match a `select` is **silently dropped** — a very common source of "why isn't my content showing" bugs.

2.4 ngProjectAs

Sometimes the node you want to project isn't naturally matchable by the child's `select` selector — most commonly because it's itself a component/directive with its own selector, or you want to project an `ng-container` (which has no tag/attribute of its own to select against).

`ngProjectAs` lets the parent tell Angular's projection engine "treat this element as if it matched selector X," purely for projection-slotting purposes. It does not add a real attribute to the DOM and does not affect the element's own component/directive matching.

```
<!-- child expects [card-header] -->
<ng-content select="[card-header]"></ng-content>
```

```
<!-- parent wants to project an ng-container as the header -->
<app-card>
  <ng-container ngProjectAs="[card-header]">
    <h2>{{ title }}</h2>
    <button (click)="close()">x</button>
  </ng-container>
</app-card>
```

Without `ngProjectAs`, the `<ng-container>` has no attributes/tag for `[card-header]` to match, so it would fall through to the wildcard slot (or be dropped).

Another classic use: projecting a child component instance into a slot that selects on a custom attribute, when you don't want to add that attribute directly to the component's host element (e.g., to avoid coupling the child component's own selector rules):

```
<app-tabs>
  <app-custom-widget ngProjectAs="[tab-extra]"></app-custom-widget>
</app-tabs>
```

2.5 Conditional / dynamic content projection

`<ng-content>` cannot be wrapped directly in a structural directive like `*ngIf` in older Angular semantics on the `<ng-content>` tag itself in some versions — but in modern Angular (v9+ Ivy), you **can** put `*ngIf` on `<ng-content>` directly, and it works because Ivy treats it as a conditionally-rendered projection point. Two patterns:

Pattern A — conditionally show/hide a whole slot:

```
<ng-content select="[card-footer]" *ngIf="showFooter"></ng-content>
```

This works in modern Ivy-based Angular. The content is projected only when `showFooter` is true; when false, the slot's content is neither rendered nor kept in a detached state — it's simply not inserted.

Pattern B — check for content presence and conditionally style a wrapper (very common real pattern):

Because you often need to know *whether* projected content exists (e.g., to hide an empty header wrapper `<div>`), you combine `@ContentChild` / `@ContentChildren` with `*ngIf`:

```
@Component({ selector: 'app-card', template: `
  <div class="card">
    <div class="card-header" *ngIf="headerContent">
      <ng-content select="[card-header]"></ng-content>
    </div>
    <div class="card-body">
      <ng-content></ng-content>
    </div>
  </div>
`})
export class CardComponent {
  @ContentChild('headerContent') headerContent?: ElementRef;
}
```

Pattern C — dynamic projection via `ngTemplateOutlet`:

When the parent needs to hand the child a *template* rather than static markup (so the child can render it zero, one, or many times, optionally with a context object), use `<ng-template>` + `TemplateRef` + `@ContentChild(TemplateRef)` + `ngTemplateOutlet`, instead of `<ng-content>`. This is the mechanism behind row templates in table/list components (e.g., a custom "item template" repeated per data item) — `ng-content` can only project each piece of content **once**, but a captured `TemplateRef` can be instantiated multiple times with different contexts.

```
<!-- parent -->
<app-list [items]="users">
  <ng-template #itemTpl let-user>
    <span>{{ user.name }}</span>
  </ng-template>
</app-list>
```

```

@Component({ selector: 'app-list', template: `
  <li *ngFor="let item of items">
    <ng-container *ngTemplateOutlet="itemTpl; context: { $implicit: item }"></ng-container>
  </li>
`})
export class ListComponent {
  @Input() items: any[] = [];
  @ContentChild(TemplateRef) itemTpl!: TemplateRef<any>;
}

```

This is a distinct feature from `<ng-content>` — it's "template projection" rather than "content projection" — but interviewers frequently ask you to contrast the two, so know it belongs in this family of techniques.

2.6 @ContentChild / @ContentChildren

These decorators query elements/directives/components **that live in the projected content** (i.e., were authored in the parent's template and passed into the host), as opposed to `@ViewChild` / `@ViewChildren`, which query elements in the component's **own template**.

```

@ContentChild(TabComponent) firstTab!: TabComponent;
@ContentChildren(TabComponent) tabs!: QueryList<TabComponent>;

```

Key rules:

- Resolved **after** `ngAfterContentInit` (single) — available by `ngAfterContentChecked` on every change-detection run.
- `@ContentChildren` returns a live `QueryList` that emits on its `.changes` Observable whenever the set of matched projected elements changes (e.g., `*ngFor` adds/removes tabs) — you must subscribe in `ngAfterContentInit` to react to later changes, because the initial `QueryList` reference itself never changes, only its contents/emissions.
- Cannot be read in the constructor or `ngOnInit` — the content hasn't been projected/initialized yet. Reading in `ngOnInit` gives `undefined` (`ContentChild`) or an empty `QueryList` (`ContentChildren`, unless `{ static: true }` — which is **not** valid for content queries that depend on `*ngIf` / `*ngFor` in projected content, since dynamic content can't be static).
- `descendants` option: by default (`descendants: true` in modern Angular), `@ContentChildren` searches all levels of nested descendants in the projected content, not just direct children. Set `{ descendants: false }` to limit to direct children of the content root only (rarely needed, but asked about in interviews).
- `read` option: to query a different token off the same element, e.g. `@ContentChild(ElementRef, { read: TemplateRef })`.

3. Code Examples

3.1 Full reusable Card component (header / body / footer slots)

```

// card.component.ts

```

```

import { Component, ContentChild, ElementRef, Input } from '@angular/core';

@Component({
  selector: 'app-card',
  standalone: true,
  template: `
    <div class="card" [class.card--elevated]="elevated">
      <div class="card__header" *ngIf="hasHeader">
        <ng-content select="[card-header], .card-title"></ng-content>
      </div>

      <div class="card__body">
        <ng-content></ng-content>
      </div>

      <div class="card__footer" *ngIf="hasFooter">
        <ng-content select="[card-footer]"></ng-content>
      </div>
    </div>
  `,
  styles: [
    .card { border: 1px solid #ddd; border-radius: 8px; overflow: hidden; }
    .card--elevated { box-shadow: 0 2px 8px rgba(0,0,0,.15); }
    .card__header { padding: 12px 16px; border-bottom: 1px solid #eee; font-weight: 600; }
    .card__body { padding: 16px; }
    .card__footer { padding: 12px 16px; border-top: 1px solid #eee; text-align: right; }
  ],
})
export class CardComponent {
  @Input() elevated = false;

  // Presence checks let us collapse header/footer wrappers when nothing was projected.
  @ContentChild('headerMarker', { read: ElementRef }) private headerMarker?: ElementRef;
  @ContentChild('footerMarker', { read: ElementRef }) private footerMarker?: ElementRef;

  get hasHeader(): boolean {
    return !!this.headerMarker;
  }
  get hasFooter(): boolean {
    return !!this.footerMarker;
  }
}

```

Note: to detect presence reliably regardless of markup shape, real-world implementations often tag a template-reference variable (`#headerMarker`) on the projected root element itself, as shown in usage below — the alternative of querying by attribute selector (`[card-header]`) works equally well when the attribute is guaranteed present.

```

<!-- usage -->
<app-card [elevated]="true">
  <ng-container ngProjectAs="[card-header]">
    <h3 #headerMarker>Invoice #4471</h3>
  </ng-container>

  <p>Amount due: $240.00</p>
  <p>Due date: 2026-08-01</p>

  <div card-footer #footerMarker>
    <button (click)="pay()">Pay now</button>
  </div>
</app-card>

```

3.2 Tabs component using @ContentChildren

```

// tab.component.ts
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-tab',
  standalone: true,
  template: `
    <div class="tab-pane" [hidden]="!active">
      <ng-content></ng-content>
    </div>
  `,
})
export class TabComponent {
  @Input() title = '';
  active = false;
}

```

```

// tabs.component.ts
import {
  AfterContentInit, Component, ContentChildren, QueryList,
} from '@angular/core';
import { TabComponent } from './tab.component';

@Component({
  selector: 'app-tabs',
  standalone: true,
  template: `
    <div class="tabs__nav">
      <button
        *ngFor="let tab of tabs; let i = index"
        [class.active]="tab.active"
        (click)="select(i)">
        {{ tab.title }}
      </button>
    </div>
  `
})

```

```

    <ng-content></ng-content>
  },
})
export class TabsComponent implements AfterContentInit {
  // Projected TabComponent instances (any nesting depth by default).
  @ContentChildren(TabComponent) tabs!: QueryList<TabComponent>;

  ngAfterContentInit(): void {
    // Activate the first tab initially.
    this.activateFirst();

    // React to tabs being added/removed dynamically (e.g. *ngFor over tab data).
    this.tabs.changes.subscribe(() => this.activateFirst());
  }

  select(index: number): void {
    this.tabs.forEach((tab, i) => (tab.active = i === index));
  }

  private activateFirst(): void {
    const list = this.tabs.toArray();
    if (list.length && !list.some(t => t.active)) {
      list[0].active = true;
    }
  }
}

```

```

<!-- usage -->
<app-tabs>
  <app-tab title="Profile">
    <p>Profile content for {{ userName }}</p>
  </app-tab>
  <app-tab title="Settings">
    <p>Settings content</p>
  </app-tab>
  <app-tab title="Billing" *ngIf="showBilling">
    <p>Billing content</p>
  </app-tab>
</app-tabs>

```

Because `*ngIf="showBilling"` sits on a projected `<app-tab>`, the `tabs` `QueryList` genuinely changes membership when `showBilling` flips — this is exactly why `tabs.changes` (not just the initial snapshot) must be subscribed to.

3.3 Modal component combining conditional projection + `ngProjectAs`

```

// modal.component.ts
import { Component, EventEmitter, Input, Output } from '@angular/core';

@Component({

```



```

selector: 'app-modal',
standalone: true,
template: `
  <div class="modal-backdrop" *ngIf="open" (click)="close.emit()">
    <div class="modal" (click)="$event.stopPropagation()">
      <header class="modal__header">
        <ng-content select="[modal-title]"></ng-content>
        <button class="modal__close" (click)="close.emit()">x</button>
      </header>

      <section class="modal__body">
        <ng-content></ng-content>
      </section>

      <footer class="modal__footer" *ngIf="showFooterSlot">
        <ng-content select="[modal-actions]"></ng-content>
      </footer>
    </div>
  </div>
`,
},
})
export class ModalComponent {
  @Input() open = false;
  @Input() showFooterSlot = true;
  @Output() close = new EventEmitter<void>();
}

```

```

<!-- usage -->
<app-modal [open]="isOpen" (close)="isOpen = false">
  <ng-container ngProjectAs="[modal-title]">
    <h2>Delete item?</h2>
  </ng-container>

  <p>This action cannot be undone.</p>

  <ng-container ngProjectAs="[modal-actions]">
    <button (click)="isOpen = false">Cancel</button>
    <button (click)="confirmDelete()">Delete</button>
  </ng-container>
</app-modal>

```

4. Internal Working

4.1 Compile-time resolution into fixed projection buckets

Ivy (Angular's current compiler/runtime) treats content projection as a **static, compile-time-known** feature, not a dynamic runtime search. When the compiler processes a component's template, every `<ng-content>` it finds is compiled into a `projectionDef` / `projection` instruction pair, roughly:

1. `ɵprojectionDef(selectors?)` — emitted once per component template (if it uses `<ng-content>` at all). It declares, in order, the list of CSS selectors used by that template's `<ng-content select="...">`

elements. This becomes a fixed array of "buckets," e.g. `[[wildcard], '[card-header]', '[card-footer]']`.

2. `ɵprojection(index, selectorIndex)` — emitted at each `<ng-content>` location, referencing which bucket (by index) should be rendered there.

At the call site (where `<app-card>...</app-card>` is used), Angular does **not** wait until render time to figure out matching. Instead, when the *parent's* view is being constructed, each top-level child node inside the `<app-card>` tags is distributed into the buckets **once**, based on the selector list the child component declared (this is why the `select` list is captured by `projectionDef` ahead of time — Angular needs to know the full set of selectors before it can bucket anything). The distribution algorithm:

- Walk the immediate child nodes of the projectable content in source order.
- For each node, test it against the selector list **in the order the selectors appear in the child template** (i.e., `<ng-content select>` document order), and place it in the first bucket whose selector matches (`ngProjectAs`, if present on that node, is used instead of the node's own tag/attributes for this test).
- A node that matches no selector falls into the wildcard bucket (bucket 0) if one exists; otherwise it is dropped (never rendered, though it still exists as a detached node reference internally — it's just never inserted into any DOM location).

Because bucketing happens once, based on the static list of selectors compiled from the template, projection is **not** a live/reactive CSS query against the DOM — you cannot, for example, dynamically change which selector a `<ng-content>` uses at runtime; the selectors are fixed at compile time (though which *node* ends up in the wildcard bucket can change over time if the actual set of projected elements changes, e.g. via `*ngFor` inside the projected content or `*ngIf` toggling a projected element in/out).

4.2 Why projected content stays bound to its declaring component

This is the single most-tested internal-mechanics fact in this chapter.

When Angular compiles `<app-card><p>{{invoiceTotal}}</p></app-card>` inside, say, `InvoiceComponent`'s template, the `<p>{{invoiceTotal}}</p>` node — including its interpolation `{{invoiceTotal}}` — is compiled as part of `InvoiceComponent`'s **view**, not `CardComponent`'s view. The compiler resolves `invoiceTotal` against `InvoiceComponent`'s class fields at compile time, generating update instructions that are only ever executed as part of `InvoiceComponent`'s change-detection pass.

`<ng-content>` in `CardComponent`'s template is purely a **DOM relocation marker** — at render time Angular takes the already-instantiated, already-bound DOM nodes/text-bindings belonging to `InvoiceComponent`'s view and inserts them at the position of `<ng-content>` inside `CardComponent`'s rendered DOM tree.

`CardComponent` never re-parses, re-compiles, or re-binds that content; it has no template-level knowledge of `invoiceTotal` at all.

Consequences:

- Projected content's expressions (interpolations, property bindings, event handlers, `*ngIf` / `*ngFor` on it) are evaluated against the **parent's** component instance (`this` inside those bindings is the parent), and they are refreshed during the **parent's** change-detection pass, in the parent's position in the CD tree — *not* as a "child" of the `CardComponent`'s CD subtree.
- This means if the parent's change detection is skipped (e.g., `OnPush` on the parent with no triggering

input/event) but the `CardComponent` re-renders for its own unrelated reasons, the projected content will **not** update, because updating it is entirely the parent's responsibility.

- Conversely, `CardComponent` can be `OnPush` and still show up-to-date projected content whenever the *parent* runs CD, because the DOM nodes are shared (moved, not copied) — `CardComponent`'s `OnPush` gate only affects `CardComponent`'s own template bindings, never the already-bound projected nodes sitting inside it.
- This is different from Angular Elements/native Shadow DOM `<slot>` "logical vs physical tree" semantics but conceptually analogous: the projected content's "logical parent" for binding/CD/DI purposes remains where it was declared.

4.3 Interaction with Dependency Injection

Because projected content is compiled as part of the parent, any directive on a projected element resolves its injected dependencies through the **parent's injector hierarchy** at the point where the content was declared, not through `CardComponent`'s injector — except that once physically inserted into `CardComponent`'s DOM subtree, `viewProviders` declared on ancestor elements *within CardComponent's own template* still apply to component/directive instances hosted by `CardComponent` itself (not to the projected content's own directives, whose injectors were fixed at their original declaration point in the element injector tree corresponding to the parent template).

5. Edge Cases & Gotchas

5.1 "My binding shows the wrong value / `this` is wrong"

The most common confusion: a developer expects a method or property referenced inside projected content to resolve against the *hosting* component (e.g., `CardComponent`), because visually the markup now lives "inside" the card. It never does — it always resolves against whichever component's template physically contains that markup in source. If `CardComponent` needs to expose data to projected content, it cannot do so via plain interpolation; it must either:

- Expose an `@Input()` that the parent binds and interpolates itself, or
- Use the `ngTemplateOutlet` + captured `TemplateRef` + `context` pattern (Section 2.5C), which explicitly hands data from child → projected template via `let-x` context variables — this is the *only* content-projection-adjacent technique that lets the host push its own data into projected markup.

5.2 `select` selector ordering and specificity

- Buckets are filled in the order `<ng-content select>` tags appear in the **child's template**, not in the order attributes appear on the projected element, and not in the order the parent writes its children.
- A node is placed into the **first** matching bucket only — if a node could match two different `select` patterns (e.g., `select="[foo]"` and a later `select="[foo].bar"`, and the node has both `foo` and `bar`), it goes to whichever selector's `<ng-content>` appears earlier in the child template. There is no "most specific selector wins" rule like real CSS specificity — it's pure document-order-of-first-match.
- If two sibling `<ng-content>` elements use the exact same `select` value, only the **first** one in the template actually receives any matching nodes; the second one will always be empty (each source node is placed into exactly one bucket, and lookup finds the first matching bucket).

- The wildcard `<ng-content>` (no `select`) always corresponds to "whatever didn't match anything else," regardless of its position in the template.

5.3 Content silently disappearing

- If the child template has `select`-based slots but no wildcard `<ng-content>`, any parent-authored markup that doesn't match one of the declared selectors is dropped — not an error, just invisible. This is a frequent "why did my extra `<div>` disappear" bug.
- Whitespace-only text nodes between elements typically get their own (usually harmless) bucket assignment, but stray *significant* text at the top level of projectable content (not wrapped in an element) can only be caught by the **wildcard** bucket — a `select` attribute/class/element selector cannot match a bare text node.

5.4 Structural directives on the projected element

- A structural directive (`*ngIf`, `*ngFor`, `*ngSwitchCase`) placed directly on the element you intend to select against **desugars to** `<ng-template>`, which changes the effective node the projection algorithm sees. E.g., `<div card-header *ngIf="cond">...</div>` desugars roughly to `<ng-template [ngIf]="cond"><div card-header>...</div></ng-template>` — the outer node offered up for projection matching is now the `<ng-template>`, which typically does **not** carry the `card-header` attribute (Ivy does preserve the projection-relevant attributes on the template in most cases for exactly this reason, but subtle cases — like matching on a class the structural directive's host doesn't propagate, or matching an *inline-style-derived* selector — can cause mismatches). The robust fix is to use `ngProjectAs` on the same element carrying the structural directive:

```
<div *ngIf="cond" ngProjectAs="[card-header]" card-header>...</div>
```

This guarantees the projection engine buckets it correctly regardless of how the structural directive desugars.

- A safer, very common pattern to sidestep this entirely: wrap in `<ng-container>` and put `ngProjectAs` + `*ngIf` on the container, with the real element(s) inside — as shown in Section 3.3.

5.5 @ContentChildren / @ContentChild timing

- Not populated in the constructor or `ngOnInit` — only from `ngAfterContentInit` onward. Reading too early gives `undefined` / an empty `QueryList`.
- `{ static: true }` is **not compatible** with content that only exists conditionally (behind `*ngIf` / `*ngFor` in the *parent's* projected markup) — Angular will throw or the query will simply be empty/incorrect, because "static" queries require the queried node to be unconditionally present at the top level of its template section. In practice, `static: true` is almost never used for content queries (it's overwhelmingly a `@ViewChild` concern); most real code doesn't set it for `@ContentChild`.
- `QueryList` is a **live** collection, but the property reference itself never changes — you must subscribe to `.changes` to react to later add/remove events (see the Tabs example, Section 3.2). A common bug is reading `this.tabs.first` once in `ngAfterContentInit` and never updating it after a `*ngFor` in the projected content adds a new tab.
- Order in a `QueryList` reflects DOM/template order of the projected elements, which is itself just the

order the parent authored them, filtered by projection bucket assignment.

- Querying a component/directive type only finds instances that were actually **instantiated as part of the projected content** — a plain HTML element with a matching attribute but no directive applied to it will not be found by `@ContentChild(someDirective)`; you'd need `@ContentChild('templateRefVar')` or `@ContentChild(ElementRef)` with a template reference variable instead.

5.6 Multiple projected instances / re-projection ("transitive" projection)

`<ng-content>` can only project a given piece of content into **one place, once**. If `CardComponent`'s own template tries to re-project its `<ng-content>` output into yet another nested component's `<ng-content>` (chained projection through multiple component layers), each layer forwards the *already-resolved* projected nodes onward — this works (and is a legitimate pattern for building layered wrapper components), but you cannot project the *same* content twice into two different locations within the same component. For "render this N times" semantics, you need the `TemplateRef` + `ngTemplateOutlet` approach instead (Section 2.5C), since a captured template can be instantiated repeatedly with different contexts, whereas `<ng-content>` nodes are moved (not cloned).

6. Interview Questions & Answers

Q1. What is `<ng-content>` and how is it different from `*ngIf` / `*ngFor`?

`<ng-content>` is a placeholder that marks where content passed in by a parent template should be rendered inside a child component's template. Unlike structural directives, it doesn't create its own embedded view or change-detection context — it's a compile-time projection marker that the Ivy compiler turns into `ɵɵprojectionDef` / `ɵɵprojection` instructions, which relocate already-instantiated parent DOM nodes into the child's render tree.

Q2. How do you project different pieces of content into different regions of a component (e.g., header vs. body vs. footer)?

Use multiple `<ng-content>` elements, each with a `select` attribute holding a CSS selector (element, class, or attribute selector). Angular assigns each top-level projected node to the first `<ng-content select>` whose selector matches it, checked in the order those `<ng-content>` tags appear in the child's own template. An unqualified `<ng-content>` with no `select` acts as the catch-all for anything left over.

Q3. What happens to content that doesn't match any `select` and there's no wildcard `<ng-content>`?

It is silently dropped — never rendered anywhere, and Angular does not throw an error or warning. This is a very common source of "my markup just vanished" bugs, so a defensive habit is to always include a wildcard `<ng-content>` unless you deliberately want to constrain what's allowed through.

Q4. What is `ngProjectAs` for, and when do you need it?

Interviewer intent: checks whether the candidate understands that `select` matching happens against the *outer* element's own tag/attributes/classes, and that some wrapper elements (like `<ng-container>`) have none of their own to match against.

`ngProjectAs` overrides what selector an element is matched against during content projection, without adding any real attribute to the DOM. It's needed whenever the element you want to project doesn't itself carry the attribute/class the child's `select` is looking for — most commonly when using `<ng-container>` (which renders no DOM element at all) as a wrapper for projected content, or when you don't want to pollute a component's actual host element with an extra projection-only attribute.

Q5. Can you conditionally project content, e.g., only show a footer slot if some condition is true?

Yes, in two complementary ways. First, you can put `*ngIf` directly on the `<ng-content>` element itself in modern Ivy Angular — the slot is projected only when the condition is true. Second, and more commonly for "collapse the wrapper `<div>` if nothing was actually passed," you combine a `*ngIf` on the *wrapping element* around `<ng-content>` with a `@ContentChild` presence check (e.g., querying by template-reference variable) so the wrapper only renders when real content exists.

Q6. If a child component's `OnPush` change detection doesn't run, will projected content inside it stay stale?

Interviewer intent: tests the core "projected content belongs to its declaring component" mental model, not surface syntax.

No — and this is the crux of understanding projection internals. Projected content's bindings are compiled as part of the *parent's* template, not the child's, so they are refreshed whenever the *parent's* change detection runs, regardless of whether the child (e.g., a `CardComponent` with `OnPush`) itself re-renders. The child's `OnPush` gate only controls the child's own template bindings; the projected DOM nodes are simply relocated into the child's rendered output and are otherwise entirely the parent's responsibility to keep current.

Q7. What's the difference between `@ContentChild` / `@ContentChildren` and `@ViewChild` / `@ViewChildren`?

`@ViewChild(ren)` queries elements/directives declared in the component's **own** template.

`@ContentChild(ren)` queries elements/directives that were passed in as **projected content** from the parent, i.e., that live between the component's opening and closing tags in the parent's template. They're populated at different lifecycle points: `ViewChild` after `ngAfterViewInit`, `ContentChild` after `ngAfterContentInit` (content is always resolved before view, so content hooks fire before view hooks).

Q8. Why might `@ContentChild` return `undefined` when read inside `ngOnInit`?

Because content projection resolution — matching and instantiating the projected nodes and populating content queries — happens after `ngOnInit` but before `ngAfterContentInit`. `ngOnInit` runs too early in the lifecycle for content queries to be populated; you must read `@ContentChild` results starting in `ngAfterContentInit` (or later hooks).

Q9. You use `@ContentChildren(TabComponent)` `tabs: QueryList<TabComponent>` and read `tabs.first` once in `ngAfterContentInit`. Later, an `*ngFor` in the parent's projected content adds a new tab, but your component doesn't see it. Why, and how do you fix it?

Interviewer intent: tests whether the candidate knows `QueryList` is a live but push-based collection, not a plain array snapshot.

`QueryList` is a live collection whose *contents* update automatically as the DOM/projected structure changes, but reading `.first` / `.toArray()` once only captures a snapshot at that moment — it does not become "reactive" on its own for values already assigned to another variable. The fix is to subscribe to `tabs.changes` (an Observable that emits whenever the `QueryList`'s membership changes) in `ngAfterContentInit`, and re-derive anything (like "which tab is active") inside that subscription so it stays current as tabs are added or removed.

Q10. You put `*ngIf` directly on a `<div>` carrying an attribute selector like `[card-header]` that a child `<ng-content select="[card-header]">` expects to match. It works in the DOM, but the projection is inconsistent or the attribute doesn't match. What's happening, and what's the fix?

Structural directives desugar into an `<ng-template>` wrapping the element (e.g., `*ngIf` becomes `<ng-template [ngIf]="...">`). The node actually offered to the projection matcher can, in certain compilations, be the template wrapper rather than the original element with its attribute, causing mismatches against

attribute/class selectors. The robust fix is to add `ngProjectAs="[card-header]"` explicitly onto the same element (or onto a wrapping `<ng-container>` that carries both the structural directive and `ngProjectAs`), which forces correct bucket assignment regardless of how the structural directive desugars.

Q11. How would you let a host component push its own data into content the parent projects (e.g., a list component that lets the parent define a custom row template, and the host supplies the actual row data)?

Plain `<ng-content>` cannot do this — projected content is compiled and bound entirely against the parent, so the host has no way to feed it data through interpolation. Instead, the parent authors an `<ng-template>` with a `let-x` context variable, the host captures that `TemplateRef` via `@ContentChild(TemplateRef)`, and the host renders it (potentially multiple times, e.g., once per data row) using `*ngTemplateOutlet="tpl; context: { $implicit: item }"`. This is "template projection," a distinct but related technique from content projection, and it's the standard pattern behind customizable row/item templates in list/table/tree components.

Q12. Two sibling `<ng-content select="[foo]">` elements exist in a component's template. What happens to nodes matching `[foo]`?

Only the first `<ng-content select="[foo]">` in template document order will ever receive matching nodes — each projected node is assigned to exactly one bucket (the first one whose selector matches it), so any subsequent `<ng-content>` with an identical (or now-redundant) selector will always render empty for that selector.

Q13. Does content projection work with Shadow DOM view encapsulation the same way as with the default `Emulated` encapsulation?

Functionally, yes — projection buckets are resolved the same way regardless of `ViewEncapsulation` mode, since bucketing happens in Angular's own Ivy instructions, independent of style encapsulation. With `ViewEncapsulation.ShadowDom`, Angular actually uses the browser's native `<slot>` element under the hood for `<ng-content>` rather than emulating projection purely in the render tree, so styling scoping follows native Shadow DOM rules (e.g., `::slotted()` for styling projected content), whereas with `Emulated / None` encapsulation Angular's own attribute-based scoping and Ivy's internal projection instructions handle it without any native `<slot>` involved.

Q14. How do you detect whether any content at all was projected into the default (wildcard) slot, so you can render a placeholder/empty-state instead?

Query the projected content via `@ContentChildren` (with a broad selector, or targeting `ElementRef` / a template reference variable placed on top-level projected nodes) and check whether the resulting `QueryList` is empty in `ngAfterContentInit` / `ngAfterContentChecked`, driving an `*ngIf` on a placeholder template. There's no built-in "isEmpty" API on `<ng-content>` itself — presence detection is always done indirectly through a content query.

7. Quick Revision Cheat Sheet

- `<ng-content>` = compile-time placeholder; projected DOM nodes are *moved*, not copied or re-rendered, and belong to the component that declared them.
- No `select` → single-slot, catches everything. With `select`, multiple slots split content by CSS selector; unqualified `<ng-content>` = wildcard catch-all for leftovers.
- Bucket assignment: first-match-wins, checked in the order `<ng-content select>` appears in the **child's** template — not CSS specificity, not parent authoring order.

- Content matching no `select` and no wildcard present = silently dropped.
- `ngProjectAs="selector"` overrides what an element is matched against for projection purposes only (no real DOM attribute added) — essential for `<ng-container>` wrappers and elements with structural directives.
- Projected content's bindings/expressions evaluate against the **parent** (declaring component); child's `OnPush` status never blocks projected content from updating, because the child doesn't own those bindings.
- `@ContentChild` / `@ContentChildren` query **projected** content; `@ViewChild` / `@ViewChildren` query the component's **own** template. Content hooks (`ngAfterContentInit` / `Checked`) fire before view hooks.
- Content queries are `undefined` / empty before `ngAfterContentInit`; never rely on them in constructor / `ngOnInit`.
- `QueryList` is live but its reference doesn't change — subscribe to `.changes` to react to dynamic add/remove of projected elements (e.g., behind `*ngFor` / `*ngIf`).
- `*ngIf` directly on `<ng-content>` works in modern Ivy Angular for conditionally including a whole slot; combine with a content-presence query to collapse empty wrapper markup.
- For host-to-projected-content data flow (rendering a parent-supplied template N times with data), use captured `TemplateRef` + `ngTemplateOutlet` + context — not `<ng-content>`, which can only project once.
- Structural directives on a projected element desugar to `<ng-template>` and can break `select` attribute/class matching — add `ngProjectAs` explicitly to guarantee correct bucketing.
- `ShadowDom` encapsulation maps `<ng-content>` to a native `<slot>`; `Emulated` / `None` handle projection purely through Ivy's internal instructions.

Created By - Durgesh Singh